

MemPalace vs. Our Pipeline

Why We Don't Need It — and What We Have Instead

NotesApp Project · April 2026

Contents

1	What MemPalace Offers
1.1	Core Value Proposition
1.2	The Hook System
1.3	The Palace Memory Store
1.4	MCP Tools
1.5	Installation Requirements
1.6	Token Economy Claims
2	Why We Don't Need It
2.1	Feature-by-Feature Comparison
2.2	The Eight Tools That Replace It
A	CLAUDE.md — Always-On Instruction Layer
B	CODING_PRINCIPLES.md
C	Auto-Memory System
D	Pre-Task Skill (/pre-task)
E	Codebase-Researcher Subagent
F	Official-Docs-Researcher Subagent
G	Domain Skills
H	Confidence Gate
3	The One Real Gap — and How to Close It
4	Summary and Recommendation

Section 1 — What MemPalace Offers

1.1 Core Value Proposition

MemPalace is a **local-first AI memory system** built for teams and individuals who have no structured memory across AI coding sessions. It targets a specific pain point: Claude 'waking up' at the start of each session with zero recollection of prior decisions, codebase patterns, or architectural context — forcing the developer to re-explain the project from scratch every time.

Its design philosophy draws from the Zettelkasten method: small cross-referenced index cards that point to each other, enabling fuzzy discovery ('remember when we talked about that idea?') rather than relying on exact keyword recall.

1.2 The Hook System (Flagship Feature)

MemPalace's most technically interesting contribution is two shell scripts that hook into Claude Code lifecycle events:

Hook	Event	What It Does	Token Cost
mempal_save_hook.sh	Stop (every 15 msgs)	Counts human messages in the JSONL transcript. When the threshold is met, runs <i>mempalace mine</i> in the background to extract topics, decisions, and verbatim quotes. In verbose mode, blocks the AI to write a diary entry first. Uses a state file to prevent infinite save loops.	Silent: zero Verbose: medium
mempal_precompact_hook.sh	PreCompact (before compression)	Fires right before Claude Code compresses the context window. Emergency-saves everything in the current transcript via <i>mempalace mine</i> . Returns {} to allow compaction to proceed. Purpose: prevent in-session discoveries from being lost when the context window shrinks.	Zero (background only)

1.3 The Palace Memory Store

- **Hierarchy:** Wings (people/projects) → Rooms (topics) → Drawers (original content)
- **Storage policy:** All content verbatim — no summarisation or paraphrasing ever occurs
- **Retrieval accuracy:** 96.6% R@5 on LongMemEval benchmarks
- **Backend:** ChromaDB vector store (default) + SQLite temporal knowledge graph
- **Disk footprint:** ~300 MB for the default embedding model
- **Privacy:** Local-first — nothing leaves the machine unless explicitly opted in

1.4 MCP Tools

MemPalace ships 29 MCP (Model Context Protocol) tools. These cover palace reads and writes, knowledge graph operations, cross-wing navigation, drawer management, and agent diary entries. Using them requires configuring MemPalace as an MCP server in Claude Code's settings.

1.5 Installation Requirements

- Python 3.9 or later
- ~300 MB disk space for the default sentence-transformer embedding model
- ChromaDB or a compatible vector store
- `pip install mempalace` followed by `mempalace init ~/projects/myapp`
- Hook scripts deployed and registered in `.claude/settings.json`

1.6 Token Economy Claims

- **Background mining (silent mode):** zero token cost — no AI involvement, pure file I/O
- **Verbose diary mode:** does consume tokens — the AI is blocked and must write before stopping
- **Semantic retrieval:** no API calls required for core search, all inference is local

In summary: MemPalace is a well-engineered solution to a real problem — AI sessions with no memory infrastructure at all. The question is whether that problem applies to us.

Section 2 — Why We Don't Need It

2.1 Feature-by-Feature Comparison

The table below maps every MemPalace capability against our existing pipeline and gives an honest verdict on whether there is any gap.

MemPalace Feature	Our Equivalent	Verdict
Cross-session memory	File-based memory at <code>~/.claude/projects/.../memory/</code> with typed files and <code>MEMORY.md</code> index, reloaded every session turn	FULLY COVERED Ours is curated, not raw
PreCompact emergency save	<code>CLAUDE.md</code> + <code>MEMORY.md</code> reload fresh every turn; foundational context survives compression	MOSTLY COVERED (see Section 3)
Semantic 'vague recall' search	Memory index is currently 4 curated files — direct reads are instant and free	OVERKILL at this scale
Token economy	Primary token cost is code generation, not memory retrieval	WRONG TARGET doesn't address our spend
Better coding sessions	Pre-task workflow, confidence gate, and skills enforce discipline — not a memory problem	WRONG TOOL for this problem
Codebase understanding	<code>codebase-researcher</code> subagent reads actual current code — cannot go stale	FULLY COVERED ours is live, not historical
Best practices alignment	<code>official-docs-researcher</code> fetches current docs; MemPalace knows nothing about <code>.NET</code>	FULLY COVERED ours is authoritative
Human-in-the-loop	Confidence gate + mandatory Phase 5 approval before any code is written	FULLY COVERED MemPalace has zero HIL

Section 2 (continued) — The Eight Tools That Replace MemPalace

A. CLAUDE.md — The Always-On Instruction Layer

What it is: A Markdown file checked into the repository root that Claude Code loads as live system context on every single session turn — automatically, without any invocation. It is the single authoritative source of truth for what the codebase expects.

What it contains:

- Full Clean Architecture layer map and responsibility boundaries
- Handler conventions: command handler order (resolve user → load untracked → ownership check → domain methods → outbox → repository → save once)
- Persistence rules: non-tracking by default, ISystemClock, ICurrentUserService — never DateTime.UtcNow
- Outbox pattern: OutboxMessage.Create with enum event types, never magic strings
- Sync conventions: per-item conflict results, delete-wins, fractional-index Position strings
- Domain facts: Notes are block-based (no content field), MeetingLink is a nullable string
- Test project structure and assertion/mock style rules
- The confidence gate: explicit rule to surface tensions rather than silently choosing

Why it beats MemPalace for codebase context: MemPalace mines raw transcripts that can become stale. CLAUDE.md is reviewed, maintained, and version-controlled. When a convention changes, the file is updated — and that update is immediately active on the next session turn. No retrieval step, no similarity search, no latency.

How to use it properly: Keep it surgical. When a new convention is established — a new DTO naming rule, a new domain fact, a new skill trigger — add it to CLAUDE.md immediately. Do not let it drift. It is wrong to add commentary or context that belongs in CODING_PRINCIPLES.md; CLAUDE.md should contain rules, not explanations.

B. CODING_PRINCIPLES.md — Deep Principles with Implementation References

What it is: A companion document covering 11 core engineering principles, each with a *Why* section and direct references to canonical implementation files and line numbers.

Principles covered:

1	Non-tracking entity retrieval by default
2	Self-documenting code (XML docs on public APIs)
3	REFACTORED: markers on changed code sections
4	Verification & uncertainty approach — never guess
5	Online documentation verification for all frameworks

6	Clean Architecture layer boundaries
7	UnitOfWork + transactional boundaries
8	Outbox pattern and reliable message processing
9	Caching strategy and cache abstraction (ICalendarCache)
10	Cache invalidation only after successful SaveChangesAsync
11	Observability: structured logging, distributed tracing

Why it beats MemPalace: Each principle links to the actual canonical file and line. MemPalace would retrieve a verbatim snippet from a past conversation — which may reference code that has since been refactored. CODING_PRINCIPLES.md is maintained by the developer, references live code, and is used as the checklist for PR reviews.

C. Auto-Memory System — Structured, Typed, Curated

What it is: File-based memory at `~/.claude/projects/.../memory/` with a `MEMORY.md` index loaded every session. Four strictly typed categories:

Type	What it stores	Current entries
user	Role, preferences, knowledge level — shapes how Claude communicates	(ongoing)
feedback	Corrections and confirmed approaches — prevents repeating mistakes AND drifting from validated choices	PR/push workflow EF migrations preference
project	Ongoing decisions, feature initiatives, absolute dates — decays fast, includes Why and How to apply	Task categories feature decision
reference	Pointers to external resources: where things live, what to look up where	CODING_PRINCIPLES.md location

Why it beats MemPalace: MemPalace saves everything verbatim — raw bash output, build errors, tool call results, all of it — creating a noisy corpus that grows without bound. Our memory system saves only what matters: decisions, corrections, confirmed patterns. Every entry includes a *Why* so future sessions can reason about edge cases, not just follow rules blindly.

How to use it properly: Save memories immediately when a preference is confirmed or a correction is made — never batch. Include the *Why*. Run `consolidate-memory` periodically to merge duplicates and prune stale entries. Do not save code patterns or file paths that can be derived by reading the codebase — those belong in `CLAUDE.md`.

Section 2 (continued) — Tools D through H

D. Pre-Task Skill (/pre-task) — Five-Phase Mandatory Research

What it is: A slash command that enforces five sequential phases before any file is touched. It is the single most important consistency mechanism in the pipeline.

Phase 1 Codebase Research	Runs the codebase-researcher subagent for the relevant feature area. Reads handler, validator, entity, repository, controller, mappings, and tests. Returns a structured terrain summary including convention tensions and ambiguities. Cannot be skipped.
Phase 2 Docs Research	If the task touches any external framework, NuGet package, SDK, or API, runs the official-docs-researcher subagent for the specific topic. Skipped only for purely internal logic with no external dependencies.
Phase 3 Conflict Resolution	If any codebase-vs-best-practice conflict was found: it is surfaced explicitly. What the codebase does, what official guidance says, what the tradeoff is. No silent compromise. User decides before proceeding.
Phase 4 Clarification	Any remaining ambiguity in the task description or terrain is raised now. Nothing is assumed. Nothing is proceeded with until questions are answered.
Phase 5 Implementation Plan	A structured report: files to create/modify, step-by-step order, patterns to apply, decisions made and why. EXPLICIT USER APPROVAL REQUIRED before any file is written.

Why it beats MemPalace for consistent sessions: MemPalace improves recall. Pre-task improves discipline. These solve orthogonal problems. Inconsistent coding sessions are caused by skipping research or silently guessing — not by forgetting prior conversations. No amount of better recall fixes skipped research.

How to use it properly: Always invoke `/pre-task <description>` at the start of every coding task. Never skip it. Never write code before Phase 5 is explicitly approved. If a task feels simple enough to skip, it almost certainly isn't.

E. Codebase-Researcher Subagent

What it is: A specialised subagent that explores a specific feature area thoroughly before coding begins. It reads all related files across every layer in sequence: handler, validator, entity, repository, controller, tests, and mappings. It returns a concise structured terrain summary, flags ambiguities and convention tensions, and proposes an implementation approach.

Why it beats MemPalace: MemPalace would retrieve prior conversation snippets about a feature — which reference the code as it was when the conversation happened. The codebase-researcher reads the actual current code. It cannot be stale. If a handler was refactored last week, the subagent sees the refactored version; MemPalace would return the old conversation about the previous version.

How to use it: It runs automatically as Phase 1 of /pre-task. Do not invoke it manually and then run pre-task separately — pre-task orchestrates it.

F. Official-Docs-Researcher Subagent

What it is: A specialised subagent that fetches and synthesises current official documentation for a specific framework, SDK, NuGet package, or external API before any implementation decision is made.

Why it beats MemPalace: MemPalace mines past conversations. If a past conversation referenced an outdated EF Core API or a deprecated MediatR pipeline behaviour, MemPalace would retrieve that outdated reference and present it as context. The official-docs-researcher always reads current documentation. This is critical for EF Core, FluentValidation, MediatR, Microsoft Entra, and Azure SDK — all of which evolve rapidly and have breaking changes between versions.

How to use it: Runs automatically as Phase 2 of /pre-task when external frameworks are involved. Also directly invocable via /research <topic> for targeted lookups outside of a coding task.

G. Domain Skills — Concentrated .NET Expertise

What they are: Nine on-demand skills that load concentrated expert knowledge for specific technical domains. Each has precise trigger conditions defined in CLAUDE.md.

Skill	Coverage
csharp-coding-standards	Records, sealed classes, Result pattern, async conventions, C# 12+ features, anti-patterns
csharp-type-design-performance	Sealed classes, readonly structs, static pure functions, collection type selection
csharp-concurrency-patterns	async/await for I/O, Channels for producer/consumer, hosted service patterns
efcore-patterns	NoTracking by default, query splitting, migration management, dedicated migration services
microsoft-extensions-dependency-injection	IServiceCollection extension methods, composable Add* patterns
microsoft-extensions-configuration	IValidateOptions, strongly-typed settings, validation on startup
ilspy-decompile	Decompile third-party NuGet assemblies when source is unavailable

simplify	Post-implementation review: catch unnecessary abstractions, redundant code, premature complexity
security-review	Auth, device provisioning, Azure Blob, Entra endpoint security review

How to use them properly: Follow CLAUDE.md trigger rules precisely. Do not invoke skills speculatively. Use `simplify` after every completed feature without exception. Use `security-review` whenever touching auth, Azure Blob, Entra, or device provisioning — no exceptions.

H. Confidence Gate — The Human-in-the-Loop Enforcer

"Do not propose or implement unless confident across: current official docs, this codebase's patterns, and architectural coherence. When codebase conventions and official best practices conflict, do not silently pick one or silently compromise. Name the tension, explain the tradeoff clearly, and resolve it with the user before writing any code."

This rule, embedded in CLAUDE.md, converts a capable AI into a disciplined collaborator. MemPalace has no HIL mechanism — it saves and retrieves, it does not reason about when human judgment is required. The confidence gate is the difference between an AI that occasionally makes hidden assumptions and one that surfaces every non-trivial choice before acting on it.

How to use it properly: Treat any violation as a session bug. If code was written without surfacing a genuine conflict, that is the failure mode to investigate and correct — not merely to fix the output of.

Section 3 — The One Real Gap and How to Close It

3.1 The Gap

The honest gap in our pipeline is the **PreCompact event**. When Claude Code compresses the context window mid-session — during a long debugging chain, a multi-file research pass, or an extended integration test investigation — any in-session discoveries that were not yet saved to memory files are lost.

- A pattern noticed in code that contradicts CLAUDE.md — not yet saved
- A domain fact surfaced during debugging — not yet saved
- A decision made mid-session about how to handle an edge case — not yet saved

MemPalace addresses this with its `mempal_precompact_hook.sh`. We currently have *nothing* wired to the PreCompact event. CLAUDE.md and MEMORY.md survive compression because they are reloaded fresh on each turn, but new in-session discoveries do not.

3.2 The Fix — Without MemPalace

We already have `anthropic-skills:consolidate-memory`. Wiring a PreCompact hook to trigger memory consolidation closes this gap with **zero new infrastructure** — no Python, no 300 MB embedding model, no ChromaDB, no palace hierarchy to maintain.

What the hook would do:

1. Fire before compaction begins
2. Prompt Claude to identify any in-session discoveries not yet saved to memory files
3. Save them as properly typed memory files (feedback, project, or reference as appropriate)
4. Update MEMORY.md with new index entries
5. Return `{}` to allow compaction to proceed

This is one addition to `.claude/settings.json` — a hooks block with a PreCompact event pointing to the consolidate-memory skill invocation. No external dependencies. No new tooling. No palace to maintain.

3.3 Sharper Stop-and-Ask Rules

The confidence gate is general and excellent. However, the sharpest human-in-the-loop moments are predictable and could be enumerated explicitly in CLAUDE.md to make HIL gates consistent rather than dependent on in-context judgment:

Data deletion or irreversible state change	Any task that deletes records, drops columns, or removes stored data. Stop and confirm before proceeding.
Ownership/auth boundary ambiguity	When it is unclear who owns an entity or which user roles can perform an operation. Stop and ask.

Two equally valid patterns in the codebase	When two feature areas implement the same concern differently with real tradeoffs. Surface the tension, do not silently pick.
Feature area with no existing tests	When touching code that has no tests and the testing level (unit vs integration) must be chosen. Stop and ask which level matches intent.

Section 4 — Summary and Recommendation

Do Not Adopt MemPalace

The infrastructure cost — Python runtime, 300 MB embedding model, ChromaDB, palace hierarchy maintenance, 29 MCP tools — is not justified. MemPalace is designed for the 'I have no memory system' starting point. Our pipeline is well past that.

Adding MemPalace would layer raw, noisy transcript dumps on top of an already well-curated structured memory system. It would introduce a Python dependency into an otherwise pure .NET workflow, create ongoing maintenance overhead for palace hierarchy, and produce retrieval noise from stale transcript content that predates refactors.

Pipeline Maturity — Where We Stand

Capability	MemPalace Target User	Our Status
Cross-session memory	None	File-based typed memory with MEMORY.md index
Codebase conventions	Absent or tribal knowledge	CLAUDE.md + CODING_PRINCIPLES.md (version-controlled)
Research workflow	Ad hoc or skipped	Mandatory 5-phase /pre-task skill
Codebase exploration	Manual file reading	codebase-researcher subagent (structured, live)
Framework knowledge	Training data assumptions	official-docs-researcher (always current)
Domain expertise	Generic	9 specialised .NET/C# skills
Human-in-the-loop	None	Confidence gate + Phase 5 approval (mandatory)
PreCompact protection	Not addressed	Gap — PreCompact hook pending (Section 3.2)

The Two Actions Worth Taking

1	Wire a PreCompact hook to consolidate-memory in .claude/settings.json. Closes the only real gap without any new infrastructure.	One settings.json change	Prevents in-session context loss before compression

2	Enumerate four specific stop-and-ask trigger categories in CLAUDE.md (deletion, auth ambiguity, pattern conflict, untested area). Makes HIL moments consistent.	~6 lines in CLAUDE.md	Consistent, predictable human-in-the-loop gates
---	---	-----------------------	---

MemPalace would help us if we had none of what is listed above. We have all of it. The only action is closing the PreCompact gap — and we can do that in minutes, without adopting a Python-based memory palace.